



Wind Damage Model

Logistic Vulnerability, BRI-Shifted Threshold and Duration-of-Load
Persistence

MKM Research Labs

Version 1.0 — 21-July-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1	Executive Summary	3
2	Model Purpose and Scope	3
2.1	Purpose	3
2.2	Scope	4
2.3	Out of Scope	4
3	Mathematical Framework	4
3.1	Vulnerability Curve	4
3.2	BRI Threshold Shift	4
3.3	Persistence	5
3.4	PRS Damage-Onset Trigger	5
3.5	Threshold Resolution	5
4	Input Parameters and Calibration	6
4.1	Calibration Status	6
4.2	Data Inputs	6
5	Implementation Details	7
5.1	Public Interface	7
5.2	Separation from Pricing	7
6	Sensitivity Analysis	7
7	Validation and Backtesting	9
7.1	Verification Approach	9
7.2	Backtesting	10
7.3	Automated Test Results	10
7.4	Test Document History	23
8	Model Limitations and Known Weaknesses	23
8.1	Remediation	23
9	Governance Validation Questions	24
9.1	Q1 — Model purpose and use	24
9.2	Q2 — Conceptual soundness	24
9.3	Q3 — Data quality	24
9.4	Q4 — Assumptions	24
9.5	Q5 — Implementation verification	24
9.6	Q6 — Sensitivity and stability	25
9.7	Q7 — Limitations and weaknesses	25
9.8	Q8 — Ongoing monitoring	25
9.9	Q9 — Governance and approval	25

10 Change History

25

1 Executive Summary

The Wind Damage Model (MKM-WD-001) converts the peak sustained wind a property experienced during a tropical-cyclone event into a damage ratio, and into the binary damage-onset trigger the Property Risk Spread (PRS) pricer consumes as a wind event count. It is the wind-side twin of the flood depth-damage model (MKM-DD-001) and deliberately mirrors its architecture: a pure scalar curve for baseline analysis, and a resilience-aware wrapper that shifts the curve using the Building Resilience Index (MKM-BRI-001).

The model has three components:

- **Severity.** A logistic vulnerability curve $DR(v) = [1 + e^{-a(v-v_{50})}]^{-1}$ maps peak sustained wind to a damage ratio. The slope a and the baseline v_{50} are configuration parameters.
- **Resilience.** The 50%-damage threshold v_{50} starts at the property's operational threshold (from the CDM hazard profile) and receives a signed, capped shift derived from the property's BRI wind sub-score and composite score. A hardened building tolerates more wind before the same damage; a poor building tolerates less.
- **Persistence.** A saturating duration-of-load factor $\Phi(t) \in [\phi_{\text{gust}}, 1]$ scales the peak-curve damage by how long the wind stayed above the damage-onset threshold. A momentary gust realises only the floor; a prolonged blow realises the full peak-curve damage. The peak-only curves are exactly the $t \rightarrow \infty$ limit, so the two families remain consistent and existing peak-only callers are unchanged.

The PRS leg is deliberately *binary*, not continuous: the spread is a frequency payout driven by the count of events where the peak wind reached the BRI-adjusted effective threshold. The continuous damage ratio is the insurance-loss view served by the wind-impact route and must not feed the spread.

Status: Development (Tier 2, RAG Amber). All three components, the CDM navigation layer, the threshold resolver, the event composer and the port stage are delivered with unit tests. Every numeric parameter is an engineering-judgement seed; the results in this document are directional, not calibrated.

Key risk: The model has never been calibrated against observed wind loss. The ± 5 m/s cap on the BRI shift is large relative to the curve's transition width, so the resilience credit is materially sensitive to a parameter that has no empirical basis — see WD-L1.

2 Model Purpose and Scope

2.1 Purpose

To price building wind resilience *pari-passu* with flood, fire and seismic resilience, so that a property's total physical-risk spread reflects wind exposure and wind hardening on the same footing as the other perils. The model supplies two distinct outputs to two distinct consumers: a damage ratio for insurance-loss and impact reporting, and a binary trigger for PRS event counting.

2.2 Scope

In scope: per-property wind damage for events generated by the coupled storm-typhoon pipeline; resolution of the operational wind threshold from the CDM; the BRI-driven shift of that threshold; the duration-of-load adjustment; and the per-event composition that joins a wind time-series to a portfolio.

2.3 Out of Scope

- Generating the wind field or the event set (MKM-TC-001) and querying it at a point (MKM-WS-001).
- Gust-factor conversion. The curve is defined on *sustained* wind; no peak-gust conversion is applied.
- Debris impact, wind-driven rain ingress and cascading failure between adjacent buildings.
- Contents, business interruption and additional living expense — the damage ratio is a structure-damage ratio.
- Directional vulnerability. The curve is isotropic; a building with a weak elevation facing the storm is treated the same as one facing away.

3 Mathematical Framework

3.1 Vulnerability Curve

The pure curve is a logistic in peak sustained wind v (m/s):

$$DR(v) = \frac{1}{1 + \exp(-a(v - v_{50}))}, \quad DR(v) = 0 \text{ for } v \leq 0,$$

with $a = 0.20 \text{ m}^{-1}\text{s}$ and baseline $v_{50} = 27.8 \text{ m/s}$. The exponent is guarded at ± 700 to avoid overflow at extreme inputs, saturating cleanly to 0 and 1. The choice of a logistic rather than the piecewise-linear form used on the flood side reflects the smoother observed onset of wind damage: there is no wind analogue of the sharp threshold a floor level imposes on flood depth.

3.2 BRI Threshold Shift

The effective threshold is the property's operational threshold translated by a signed resilience term:

$$v_{50}^{\text{eff}} = v_{50}^{\text{thr}} + \text{clip}\left(\alpha_w \ln \frac{B_w}{\mu_w} + \beta_c \ln \frac{B_c}{\mu_c}, \pm \delta_{\text{max}}\right),$$

where B_w is the BRI wind sub-score, B_c the BRI composite score, $\mu_w = \mu_c = 0.50$ the neutral references, $\alpha_w = 6.0 \text{ m/s}$ and $\beta_c = 3.0 \text{ m/s}$ per log-unit, and $\delta_{\text{max}} = 5.0 \text{ m/s}$ the cap. Scores are floored at 10^{-6} before the logarithm.

The structure is the direct analogue of the flood-side *stilt*, in m/s rather than metres. It is log-differential so that the credit is proportional to the *ratio* of the score to the neutral reference: moving from 0.25 to 0.50 earns the same shift as moving from 0.50 to 1.00. The wind sub-score carries twice

the weight of the composite, reflecting that wind-specific measures (roof-to-wall connection, opening protection, roof covering) bear more directly on wind damage than general building quality.

Passing `None` for either score substitutes the neutral reference, contributing zero from that term. This is a deliberate no-information default, not an error path.

3.3 Persistence

The duration-of-load multiplier is a saturating exponential in the hours t the sustained wind stayed above the damage-onset threshold:

$$\Phi(t) = \phi_{\text{gust}} + (1 - \phi_{\text{gust}})(1 - e^{-t/\tau}), \quad \Phi(t \leq 0) = \phi_{\text{gust}},$$

with $\phi_{\text{gust}} = 0.45$ and $\tau = 4.0$ h. The full damage ratio is the product

$$DR = DR_{\text{peak}}(v; v_{50}^{\text{eff}}) \Phi(t).$$

The rationale is physical: fastener fatigue, progressive covering loss and water ingress all accumulate with exposure time, so a single momentary gust at a given speed is less destructive than hours at the same speed. The floor ϕ_{gust} encodes that even an instantaneous peak does real damage.

3.4 PRS Damage-Onset Trigger

The PRS leg counts events, not losses. The trigger is

$$\mathbf{1}[v_{\text{peak}} \geq v_{50}^{\text{eff}}],$$

falling back to the raw operational threshold when the effective threshold is absent from a damage row (legacy rows written before the field existed). Because the threshold is BRI-adjusted, a resilient building fires the trigger less readily than a poor building exposed to the same wind — which is precisely the mechanism by which resilience earns spread. Wind-without-flood is therefore possible, and its frequency is governed by where the BRI-adjusted exceedance sits relative to the coupled storm's wind.

3.5 Threshold Resolution

The operational threshold is resolved from the CDM in a fixed precedence order:

1. `WindThresholdMinorMps` — damage onset, preferred.
2. `WindThresholdMajorMps` — severe, used as fallback.
3. `WindThresholdKph` — legacy, divided by 3.6.
4. The configured default (100 km/h $\rightarrow$ 27.8 m/s).

The unit convention is a known historical hazard in this model: the CDM originally carried km/h and now carries m/s. All internal mathematics is in m/s; the resolver is the single conversion point.

4 Input Parameters and Calibration

Table 1: Model parameters

Symbol	Constant	Value	Source
a	WIND_SIGMOID_A_PER_MS	0.20	config/damage.py
v_{50}	WIND_V50_BASE_MS	27.8	config/damage.py
—	DEFAULT_WIND_THRESHOLD_KPH	100.0	config/damage.py
ϕ_{gust}	WIND_PERSISTENCE_GUST_FLOOR	0.45	config/damage.py
τ	WIND_PERSISTENCE_TAU_H	4.0	config/damage.py
α_w	BRI_WIND_ALPHA_MS	6.0	config/bri.py
β_c	BRI_COMPOSITE_BETA_MS	3.0	config/bri.py
μ_w	BRI_WIND_REFERENCE	0.50	config/bri.py
μ_c	BRI_COMPOSITE_REFERENCE	0.50	config/bri.py
δ_{max}	WIND_V50_SHIFT_MAX_MS	5.0	config/bri.py

4.1 Calibration Status

None of the ten parameters above is empirically calibrated. Each is an engineering-judgement seed chosen so that the curve behaves sensibly across the range of winds the coupled pipeline produces: damage onset around gale force, half damage near 28 m/s (roughly 100 km/h), and saturation by 50 m/s. The persistence parameters are anchored on the observation that a typical tropical cyclone subjects a point to damaging winds for a few hours, so $\tau = 4$ h places realistic events on the steep part of Φ rather than at either asymptote.

4.2 Data Inputs

Table 2: Per-property inputs

Input	Source	Missing-value behaviour
Peak sustained wind (m/s)	MKM-WS-001 / windts	required
Duration above threshold (h)	windts	$\leq 0 \Rightarrow$ gust floor
Operational threshold	CDM hazard profile	configured default
BRI wind sub-score	resilience CDM	neutral 0.50
BRI composite score	resilience CDM	neutral 0.50

5 Implementation Details

Table 3: Implementation map (`src/models/winddamage/`)

Module	Responsibility
<code>vulnerability.py</code>	the four curves and $\Phi(t)$
<code>bri_shift.py</code>	signed, capped v_{50} shift
<code>threshold.py</code>	CDM threshold $\rightarrow$ m/s; PRS trigger
<code>cdm.py</code>	property-record navigation
<code>extraction.py</code>	windts <code>EVT-*.json</code> $\rightarrow$ per-property peaks
<code>event.py</code>	per-event composer: windts + portfolio $\rightarrow$ damage JSON

5.1 Public Interface

Four curve functions are exposed, forming a two-by-two grid over $\{\text{pure, BRI-aware}\} \times \{\text{peak-only, persistence-aware}\}$:

	Peak only	With persistence
Pure curve	<code>scalar_peak_damage</code>	<code>scalar_wind_damage</code>
BRI-aware	<code>bri_peak_damage</code>	<code>bri_wind_damage</code>

The peak-only column is the $t \rightarrow \infty$ limit of the persistence column, so the grid is internally consistent by construction rather than by convention.

5.2 Separation from Pricing

`is_prs_wind` lives in this model, but it returns a boolean only. No part of this model converts damage to currency or to basis points; that is the PRS pricer’s responsibility. The separation matters because the continuous damage ratio and the binary trigger must not be conflated — feeding the damage ratio into the spread would double-count severity that the spread’s frequency formulation already handles.

6 Sensitivity Analysis

Generated by calling the model directly (`python -m docs.models.sensitivities.generate_all --model winddamage`). The tables demonstrate the three axes — severity, resilience and duration — and the resulting PRS trigger behaviour.

The material observations:

- **The resilience credit is large.** At 27.8 m/s an NR-grade building takes 0.85 damage against 0.24 for AA — a factor of 3.5. The spread narrows at both tails, as the logistic saturates.
- **The credit is widest in the mid-range.** Below 15 m/s and above 55 m/s, grade barely matters. Between 20 and 40 m/s it dominates. This is the designed behaviour: resilience should not be credited for surviving a breeze, nor for surviving a wind that destroys everything.

- **The trigger is grade-discriminating exactly where it should be.** At 30 m/s, NR and B fire while A and AA do not; by 35 m/s all four fire. The discriminating band is roughly 19–33 m/s, which is where the effective thresholds spread across grades.
- **Persistence is material but bounded.** Φ ranges from 0.45 at zero duration to 0.997 at 24 h, so duration can vary realised damage by slightly more than a factor of two at fixed severity.

Table 4: Pure vulnerability curve: damage ratio by peak sustained wind, at the configured base $v_{50} = 27.8$ m/s and slope $a = 0.2$ m⁻¹s.

Peak wind (m/s)	Damage ratio
0.0000	0.0000
10.0	0.0277
17.5	0.1130
22.0	0.2387
27.8	0.5000
33.0	0.7389
40.0	0.9198
50.0	0.9883
60.0	0.9984
70.0	0.9998

Table 5: Signed v_{50} shift (m/s) by BRI wind sub-score and composite score, capped at ± 12.0 m/s. Positive shifts move the curve rightward (the building tolerates more wind before damage).

BRI wind score	Composite NR (0.190)	Composite B (0.500)	Composite A (0.745)	Composite AA (0.997)
0.1900	-8.71	-5.81	-4.61	-3.51
0.5000	-2.90	0.0000	1.20	2.30
0.7450	-0.5100	2.39	3.59	4.69
0.9350	0.8530	3.76	4.95	5.85

Table 6: BRI-adjusted damage ratio by resilience grade at a common operational threshold of 27.8 m/s (wind sub-score and composite both set to the grade mid-point).

Peak wind (m/s)	NR grade	B grade	A grade	AA grade
0.0000	0.0000	0.0000	0.0000	0.0000
10.0	0.1396	0.0277	0.0137	0.0091
17.5	0.4211	0.1130	0.0585	0.0397
22.0	0.6414	0.2387	0.1326	0.0922
27.8	0.8509	0.5000	0.3279	0.2448
33.0	0.9417	0.7389	0.5799	0.4783
40.0	0.9850	0.9198	0.8484	0.7881
50.0	0.9979	0.9883	0.9764	0.9649
60.0	0.9997	0.9984	0.9967	0.9951
70.0	1.00	0.9998	0.9996	0.9993

Table 7: Persistence multiplier $\Phi(t)$ by hours above the damage-onset threshold ($\phi_{gust} = 0.45$, $\tau = 4.0$ h). A momentary gust realises only the floor; a prolonged blow saturates at the full peak-curve damage.

Duration above threshold (h)	$\Phi(t)$
0.0000	0.4500
0.5000	0.5146
1.00	0.5717
2.00	0.6664
4.00	0.7977
6.00	0.8773
8.00	0.9256
12.0	0.9726
24.0	0.9986

Table 8: Persistence-aware damage ratio (severity $\times$ duration) for a B-grade building at a 27.8 m/s operational threshold.

Peak wind (m/s)	$t = 0$ h	$t = 1$ h	$t = 2$ h	$t = 4$ h	$t = 8$ h	$t = 24$ h
22.0	0.1074	0.1364	0.1590	0.1904	0.2209	0.2383
27.8	0.2250	0.2858	0.3332	0.3988	0.4628	0.4993
33.0	0.3325	0.4224	0.4924	0.5894	0.6839	0.7378
40.0	0.4139	0.5258	0.6130	0.7337	0.8514	0.9186
50.0	0.4448	0.5650	0.6586	0.7884	0.9148	0.9870
60.0	0.4493	0.5707	0.6653	0.7964	0.9241	0.9970

Table 9: PRS damage-onset trigger by resilience grade. The trigger fires when the peak sustained wind reaches the BRI-adjusted effective threshold, so a resilient building fires less readily at the same wind.

Grade	$v_{50,eff}$ (m/s)	30 m/s	35 m/s	40 m/s	45 m/s
NR	19.1	yes	yes	yes	yes
B	27.8	yes	yes	yes	yes
A	31.4	no	yes	yes	yes
AA	33.4	no	yes	yes	yes

7 Validation and Backtesting

7.1 Verification Approach

Verified by unit test against calibration-independent properties:

- **Bounds.** Every curve returns a value in $[0, 1]$ for all inputs, including extreme and negative winds.
- **Monotonicity.** Damage is non-decreasing in wind and in duration, and non-increasing in BRI score.

- **Midpoint identity.** $DR(v_{50}^{\text{eff}}) = 0.5$ exactly, for any threshold and any BRI pair.
- **Neutral BRI.** Scores at the reference 0.50 produce a zero shift; **None** behaves identically to the reference.
- **Cap.** The shift never exceeds $\pm\delta_{\text{max}}$ regardless of score, including at the log floor.
- **Persistence limits.** $\Phi(0) = \phi_{\text{gust}}$ and $\Phi(t) \rightarrow 1$; the persistence-aware curves converge to the peak-only curves as t grows.
- **Threshold precedence.** Each of the four resolution branches is exercised, including the legacy km/h conversion.
- **Trigger fallback.** `is_prs_wind` falls back to the raw threshold when the effective threshold is absent, and returns **False** rather than raising on malformed rows.

7.2 Backtesting

No backtest has been performed. A meaningful backtest requires paired observations of peak sustained wind at the property and realised structural loss, which the platform does not currently hold. This is remediation item WD-R1 and is a precondition of MRC sign-off.

7.3 Automated Test Results

Table 10: Test Results Summary — Wind Damage (MKM-WD-001)

Total Tests	248
Passed	248
Failed	0
Skipped	0
Duration	0.05s

Table 11: Detailed Test Results — Wind Damage

Test Description	Acceptance Criteria	Result	Time
Different thresholds produce different damage at same peak	<code>weak > strong</code>	Pass	0.000s
Different thresholds produce different damage at same peak	<code>weak > strong</code>	Pass	0.000s
High bri reduces damage	<code>hardened < baseline</code>	Pass	0.000s
High bri reduces damage	<code>hardened < baseline</code>	Pass	0.000s
Low bri increases damage	<code>vulnerable > baseline</code>	Pass	0.000s
Low bri increases damage	<code>vulnerable > baseline</code>	Pass	0.000s
Negative wind yields zero	<code>scalar_peak_damage(-5.0) == 0.0;</code> <code>bri_peak_damage(-5.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Negative wind yields zero	<code>scalar_peak_damage(-5.0) == 0.0;</code> <code>bri_peak_damage(-5.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
No bri scores matches pure curve at threshold	<code>v == pytest.approx(0.5, abs=1e-9)</code>	Pass	0.000s
No bri scores matches pure curve at threshold	<code>v == pytest.approx(0.5, abs=1e-9)</code>	Pass	0.000s
Only composite score supplied uses neutral wind	<code>v == pytest.approx(expected, abs=1e-12)</code>	Pass	0.000s
Only composite score supplied uses neutral wind	<code>v == pytest.approx(expected, abs=1e-12)</code>	Pass	0.000s
Only wind score supplied uses neutral composite	<code>math.isfinite(v);</code> <code>v == pytest.approx(expected, abs=1e-12)</code>	Pass	0.000s
Only wind score supplied uses neutral composite	<code>math.isfinite(v);</code> <code>v == pytest.approx(expected, abs=1e-12)</code>	Pass	0.000s
Reference bri scores have no effect	<code>a == pytest.approx(b, abs=1e-9)</code>	Pass	0.000s
Reference bri scores have no effect	<code>a == pytest.approx(b, abs=1e-9)</code>	Pass	0.000s
Shift consistent with bri v50 shift	<code>a == pytest.approx(b, abs=1e-12)</code>	Pass	0.000s
Shift consistent with bri v50 shift	<code>a == pytest.approx(b, abs=1e-12)</code>	Pass	0.000s
Zero wind yields zero damage	<code>scalar_peak_damage(0.0) == 0.0;</code> <code>bri_peak_damage(0.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Zero wind yields zero damage	<code>scalar_peak_damage(0.0) == 0.0;</code> <code>bri_peak_damage(0.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Above reference gives positive shift	<code>s > 0.0</code>	Pass	0.000s
Above reference gives positive shift	<code>s > 0.0</code>	Pass	0.000s
Analytical value at 2x reference	<code>s == pytest.approx(expected, abs=1e-9)</code>	Pass	0.000s
Analytical value at 2x reference	<code>s == pytest.approx(expected, abs=1e-9)</code>	Pass	0.000s
At references shift is zero	<code>s == pytest.approx(0.0, abs=1e-12)</code>	Pass	0.000s
At references shift is zero	<code>s == pytest.approx(0.0, abs=1e-12)</code>	Pass	0.000s
Below reference gives negative shift	<code>s < 0.0</code>	Pass	0.000s
Below reference gives negative shift	<code>s < 0.0</code>	Pass	0.000s
Composite alone moves the shift	<code>s == pytest.approx(expected, abs=1e-9)</code>	Pass	0.000s
Composite alone moves the shift	<code>s == pytest.approx(expected, abs=1e-9)</code>	Pass	0.000s
Monotonic in composite score	<code>a < b < c</code>	Pass	0.000s
Monotonic in composite score	<code>a < b < c</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Monotonic in wind score	<code>a < b < c</code>	Pass	0.000s
Monotonic in wind score	<code>a < b < c</code>	Pass	0.000s
Shift capped above	<code>s <= WIND_V50_SHIFT_MAX_MS</code>	Pass	0.000s
Shift capped above	<code>s <= WIND_V50_SHIFT_MAX_MS</code>	Pass	0.000s
Shift capped below	<code>s >= -WIND_V50_SHIFT_MAX_MS</code>	Pass	0.000s
Shift capped below	<code>s >= -WIND_V50_SHIFT_MAX_MS</code>	Pass	0.000s
Zero score is clamped not undefined	<code>math.isfinite(s);</code> <code>s ==</code> <code>pytest.approx(-WIND_V50_SHIFT_MAX_MS,</code> <code>abs=1e-9)</code>	Pass	0.000s
Zero score is clamped not undefined	<code>math.isfinite(s);</code> <code>s ==</code> <code>pytest.approx(-WIND_V50_SHIFT_MAX_MS,</code> <code>abs=1e-9)</code>	Pass	0.000s
Equals bri peak times persistence	<code>bri_wind_damage(v, t, thr, 0.7, 0.6)</code> <code>== pytest.approx(...</code>	Pass	0.000s
Equals bri peak times persistence	<code>bri_wind_damage(v, t, thr, 0.7, 0.6)</code> <code>== pytest.approx(...</code>	Pass	0.000s
High bri still reduces damage with persistence	<code>hardened < baseline</code>	Pass	0.000s
High bri still reduces damage with persistence	<code>hardened < baseline</code>	Pass	0.000s
Persistence raises damage with duration	<code>long > short</code>	Pass	0.000s
Persistence raises damage with duration	<code>long > short</code>	Pass	0.000s
Saturates to bri peak	<code>bri_wind_damage(v, 10_000.0, thr) ==</code> <code>pytest.approx(...</code>	Pass	0.000s
Saturates to bri peak	<code>bri_wind_damage(v, 10_000.0, thr) ==</code> <code>pytest.approx(...</code>	Pass	0.000s
Zero wind yields zero	<code>scalar_wind_damage(0.0, 5.0) ==</code> <code>0.0;</code> <code>bri_wind_damage(0.0, 5.0,</code> <code>threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Zero wind yields zero	<code>scalar_wind_damage(0.0, 5.0) ==</code> <code>0.0;</code> <code>bri_wind_damage(0.0, 5.0,</code> <code>threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Bri scores shift v50	<code>d["v_50_eff_ms"] > d["threshold_ms"];</code> <code>d["damage_ratio"] < baseline</code>	Pass	0.000s
Bri scores shift v50	<code>d["v_50_eff_ms"] > d["threshold_ms"];</code> <code>d["damage_ratio"] < baseline</code>	Pass	0.000s
Low bri increases damage	<code>d.vuln["damage_ratio"] ></code> <code>d.base["damage_ratio"]</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Low bri increases damage	<code>d.vuln["damage_ratio"] ></code> <code>d.base["damage_ratio"]</code>	Pass	0.000s
Missing threshold uses default	<code>d["threshold_ms"] ==</code> <code>DEFAULT.WIND.THRESHOLD_KPH / 3.6</code>	Pass	0.000s
Missing threshold uses default	<code>d["threshold_ms"] ==</code> <code>DEFAULT.WIND.THRESHOLD_KPH / 3.6</code>	Pass	0.000s
One property one event	<code>payload["event_id"] == "EVT-0001";</code> <code>payload["scenario_family"] ==</code> <code>"moderate" (+6 more)</code>	Pass	0.000s
One property one event	<code>payload["event_id"] == "EVT-0001";</code> <code>payload["scenario_family"] ==</code> <code>"moderate" (+6 more)</code>	Pass	0.000s
Property not in windts omitted	<code>ids == ["PROP-a"]</code>	Pass	0.000s
Property not in windts omitted	<code>ids == ["PROP-a"]</code>	Pass	0.000s
Records without id are skipped	<code>len(payload["damages"]) == 1</code>	Pass	0.000s
Records without id are skipped	<code>len(payload["damages"]) == 1</code>	Pass	0.000s
Counts intervals where value exceeds threshold	<code>duration_above_threshold(t, v, 30.0)</code> <code>== pytest.approx(2.0)</code>	Pass	0.000s
Counts intervals where value exceeds threshold	<code>duration_above_threshold(t, v, 30.0)</code> <code>== pytest.approx(2.0)</code>	Pass	0.000s
Empty or single sample is zero	<code>duration_above_threshold([],</code> <code>[], 1.0) == 0.0;</code> <code>duration_above_threshold([0.0],</code> <code>[99.0], 1.0) == 0.0</code>	Pass	0.000s
Empty or single sample is zero	<code>duration_above_threshold([],</code> <code>[], 1.0) == 0.0;</code> <code>duration_above_threshold([0.0],</code> <code>[99.0], 1.0) == 0.0</code>	Pass	0.000s
Zero when never exceeds	<code>duration_above_threshold([0.0, 1.0,</code> <code>2.0], [5.0, 6.0, 7.0])...</code>	Pass	0.000s
Zero when never exceeds	<code>duration_above_threshold([0.0, 1.0,</code> <code>2.0], [5.0, 6.0, 7.0])...</code>	Pass	0.000s
Both present	<code>wind == 0.6214; comp == 0.6176</code>	Pass	0.000s
Both present	<code>wind == 0.6214; comp == 0.6176</code>	Pass	0.000s
Empty dict returns none none	<code>extract_bri_scores({}) == (None,</code> <code>None)</code>	Pass	0.000s
Empty dict returns none none	<code>extract_bri_scores({}) == (None,</code> <code>None)</code>	Pass	0.000s
Garbage score value becomes none	<code>wind is None; comp == 0.6176</code>	Pass	0.000s
Garbage score value becomes none	<code>wind is None; comp == 0.6176</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Missing ratings block returns none none	<code>(wind, comp) == (None, None)</code>	Pass	0.000s
Missing ratings block returns none none	<code>(wind, comp) == (None, None)</code>	Pass	0.000s
Only composite present	<code>wind is None; comp == 0.6176</code>	Pass	0.000s
Only composite present	<code>wind is None; comp == 0.6176</code>	Pass	0.000s
Only wind present	<code>wind == 0.6214; comp is None</code>	Pass	0.000s
Only wind present	<code>wind == 0.6214; comp is None</code>	Pass	0.000s
Commercial asset location fallback	<code>extract_lon_lat(rec) == (105.837, 21.067)</code>	Pass	0.000s
Commercial asset location fallback	<code>extract_lon_lat(rec) == (105.837, 21.067)</code>	Pass	0.000s
Garbage coords become none	<code>extract_lon_lat(rec) == (None, None)</code>	Pass	0.000s
Garbage coords become none	<code>extract_lon_lat(rec) == (None, None)</code>	Pass	0.000s
Location not dict returns none none	<code>extract_lon_lat({"PropertyHeader": {"Location": "x"}}) == ...</code>	Pass	0.000s
Location not dict returns none none	<code>extract_lon_lat({"PropertyHeader": {"Location": "x"}}) == ...</code>	Pass	0.000s
Missing location returns none none	<code>extract_lon_lat({}) == (None, None)</code>	Pass	0.000s
Missing location returns none none	<code>extract_lon_lat({}) == (None, None)</code>	Pass	0.000s
Property header location preferred over commercial	<code>extract_lon_lat(rec) == (-0.1, 51.5)</code>	Pass	0.000s
Property header location preferred over commercial	<code>extract_lon_lat(rec) == (-0.1, 51.5)</code>	Pass	0.000s
Returns lon lat	<code>extract_lon_lat(rec) == (-0.1, 51.5)</code>	Pass	0.000s
Returns lon lat	<code>extract_lon_lat(rec) == (-0.1, 51.5)</code>	Pass	0.000s
Commercial asset header path	<code>extract_property_id(rec) == "CPROP-08ac3589"</code>	Pass	0.000s
Commercial asset header path	<code>extract_property_id(rec) == "CPROP-08ac3589"</code>	Pass	0.000s
Empty dict returns none	<code>extract_property_id({}) is None</code>	Pass	0.000s
Empty dict returns none	<code>extract_property_id({}) is None</code>	Pass	0.000s
Fallback to top level header	<code>extract_property_id(rec) == "CPROP-abc"</code>	Pass	0.000s
Fallback to top level header	<code>extract_property_id(rec) == "CPROP-abc"</code>	Pass	0.000s
Missing returns none	<code>extract_property_id(rec) is None; extract.wind.threshold_kph(rec) is None</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Missing returns none	<code>extract.property_id(rec)</code> is None; <code>extract.wind.threshold_kph(rec)</code> is None	Pass	0.000s
Property header path	<code>extract.property_id(_record())</code> == "PROP-test-001"	Pass	0.000s
Property header path	<code>extract.property_id(_record())</code> == "PROP-test-001"	Pass	0.000s
Property header preferred over commercial asset	<code>extract.property_id(rec)</code> == "PROP-test-001"	Pass	0.000s
Property header preferred over commercial asset	<code>extract.property_id(rec)</code> == "PROP-test-001"	Pass	0.000s
Garbage value returns none	<code>extract.wind.threshold_kph(rec)</code> is None	Pass	0.000s
Garbage value returns none	<code>extract.wind.threshold_kph(rec)</code> is None	Pass	0.000s
Int value cast to float	<code>extract.wind.threshold_kph(rec)</code> == 100.0; <code>isinstance(extract.wind.threshold_kph(rec), float)</code>	Pass	0.000s
Int value cast to float	<code>extract.wind.threshold_kph(rec)</code> == 100.0; <code>isinstance(extract.wind.threshold_kph(rec), float)</code>	Pass	0.000s
Missing protection measures returns none	<code>extract.wind.threshold_kph({})</code> is None	Pass	0.000s
Missing protection measures returns none	<code>extract.wind.threshold_kph({})</code> is None	Pass	0.000s
Missing returns none	<code>extract.property_id(rec)</code> is None; <code>extract.wind.threshold_kph(rec)</code> is None	Pass	0.000s
Missing returns none	<code>extract.property_id(rec)</code> is None; <code>extract.wind.threshold_kph(rec)</code> is None	Pass	0.000s
Present returns float	<code>extract.wind.threshold_kph(_record())</code> == 120.0	Pass	0.000s
Present returns float	<code>extract.wind.threshold_kph(_record())</code> == 120.0	Pass	0.000s
Falls back to mps times 3 6	<code>extract.wind.threshold_kph(rec)</code> == 36.0	Pass	0.000s
Falls back to mps times 3 6	<code>extract.wind.threshold_kph(rec)</code> == 36.0	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Mps fallback garbage returns none	<code>extract_wind_threshold_kph(rec)</code> is None	Pass	0.000s
Mps fallback garbage returns none	<code>extract_wind_threshold_kph(rec)</code> is None	Pass	0.000s
Kph garbage returns none	<code>extract_wind_threshold_mps(rec)</code> is None	Pass	0.000s
Kph garbage returns none	<code>extract_wind_threshold_mps(rec)</code> is None	Pass	0.000s
Legacy kph converted	<code>extract_wind_threshold_mps(rec) ==</code> 120.0 / 3.6	Pass	0.000s
Legacy kph converted	<code>extract_wind_threshold_mps(rec) ==</code> 120.0 / 3.6	Pass	0.000s
Minor garbage falls through to major	<code>extract_wind_threshold_mps(rec) ==</code> 69.44	Pass	0.000s
Minor garbage falls through to major	<code>extract_wind_threshold_mps(rec) ==</code> 69.44	Pass	0.000s
Minor preferred over major	<code>extract_wind_threshold_mps(rec) ==</code> 55.56	Pass	0.000s
Minor preferred over major	<code>extract_wind_threshold_mps(rec) ==</code> 55.56	Pass	0.000s
Mps garbage falls through to kph	<code>extract_wind_threshold_mps(rec) ==</code> 10.0	Pass	0.000s
Mps garbage falls through to kph	<code>extract_wind_threshold_mps(rec) ==</code> 10.0	Pass	0.000s
No fields returns none	<code>extract_wind_threshold_mps({})</code> is None	Pass	0.000s
No fields returns none	<code>extract_wind_threshold_mps({})</code> is None	Pass	0.000s
Preferred mps field	<code>extract_wind_threshold_mps(rec) ==</code> 33.0	Pass	0.000s
Preferred mps field	<code>extract_wind_threshold_mps(rec) ==</code> 33.0	Pass	0.000s
Non dict intermediate returns none	<code>extract_property_id({"PropertyHeader": "oops"})</code> is None	Pass	0.000s
Non dict intermediate returns none	<code>extract_property_id({"PropertyHeader": "oops"})</code> is None	Pass	0.000s
Empty row returns false	<code>is_prs_wind({})</code> is False	Pass	0.000s
Empty row returns false	<code>is_prs_wind({})</code> is False	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Falls back to threshold when v50 eff absent	is_prs_wind({"peak_sustained_ms": 35.0, "threshold_ms": 3...; is_prs_wind({"peak_sustained_ms": 25.0, "threshold_ms": 3...	Pass	0.000s
Falls back to threshold when v50 eff absent	is_prs_wind({"peak_sustained_ms": 35.0, "threshold_ms": 3...; is_prs_wind({"peak_sustained_ms": 25.0, "threshold_ms": 3...	Pass	0.000s
High bri threshold suppresses trigger	is_prs_wind({"peak_sustained_ms": wind, "threshold_ms": 6...; is_prs_wind({"peak_sustained_ms": wind, "threshold_ms": 3...	Pass	0.000s
High bri threshold suppresses trigger	is_prs_wind({"peak_sustained_ms": wind, "threshold_ms": 6...; is_prs_wind({"peak_sustained_ms": wind, "threshold_ms": 3...	Pass	0.000s
Missing peak returns false	is_prs_wind({"threshold_ms": 30.0}) is False	Pass	0.000s
Missing peak returns false	is_prs_wind({"threshold_ms": 30.0}) is False	Pass	0.000s
Missing threshold returns false	is_prs_wind({"peak_sustained_ms": 55.0}) is False	Pass	0.000s
Missing threshold returns false	is_prs_wind({"peak_sustained_ms": 55.0}) is False	Pass	0.000s
Non numeric v50 eff returns false	is_prs_wind(row) is False	Pass	0.000s
Non numeric v50 eff returns false	is_prs_wind(row) is False	Pass	0.000s
Non numeric values return false	is_prs_wind({"peak_sustained_ms": "fast", "threshold_ms": ...	Pass	0.000s
Non numeric values return false	is_prs_wind({"peak_sustained_ms": "fast", "threshold_ms": ...	Pass	0.000s
Peak above threshold triggers	is_prs_wind({"peak_sustained_ms": 55.0, "threshold_ms": 3...	Pass	0.000s
Peak above threshold triggers	is_prs_wind({"peak_sustained_ms": 55.0, "threshold_ms": 3...	Pass	0.000s
Peak below threshold does not trigger	is_prs_wind({"peak_sustained_ms": 20.0, "threshold_ms": 3...	Pass	0.000s
Peak below threshold does not trigger	is_prs_wind({"peak_sustained_ms": 20.0, "threshold_ms": 3...	Pass	0.000s
Peak equal threshold triggers	is_prs_wind({"peak_sustained_ms": 30.0, "threshold_ms": 3...	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Peak equal threshold triggers	<code>is_prs.wind({"peak_sustained_ms": 30.0, "threshold_ms": 3...})</code>	Pass	0.000s
V50 eff boundary is inclusive	<code>is_prs.wind(row) is True</code>	Pass	0.000s
V50 eff boundary is inclusive	<code>is_prs.wind(row) is True</code>	Pass	0.000s
V50 eff fires when raw threshold would not	<code>is_prs.wind(row) is True</code>	Pass	0.000s
V50 eff fires when raw threshold would not	<code>is_prs.wind(row) is True</code>	Pass	0.000s
V50 eff is preferred over raw threshold	<code>is_prs.wind(row) is False</code>	Pass	0.000s
V50 eff is preferred over raw threshold	<code>is_prs.wind(row) is False</code>	Pass	0.000s
100 kph is 27 78 ms	<code>kph_to.ms(100.0) == pytest.approx(100.0 / 3.6, abs=1e-9)</code>	Pass	0.000s
100 kph is 27 78 ms	<code>kph_to.ms(100.0) == pytest.approx(100.0 / 3.6, abs=1e-9)</code>	Pass	0.000s
Kmh to ms factor	<code>KMH.TO.MS == pytest.approx(1.0 / 3.6, abs=1e-12)</code>	Pass	0.000s
Kmh to ms factor	<code>KMH.TO.MS == pytest.approx(1.0 / 3.6, abs=1e-12)</code>	Pass	0.000s
Zero kph	<code>kph_to.ms(0.0) == 0.0</code>	Pass	0.000s
Zero kph	<code>kph_to.ms(0.0) == 0.0</code>	Pass	0.000s
Accepts string path	<code>result.peaks_by_property == {"PROP-a": 32.4, "PROP-b": 18.1}</code>	Pass	0.001s
Accepts string path	<code>result.peaks_by_property == {"PROP-a": 32.4, "PROP-b": 18.1}</code>	Pass	0.001s
Empty property windts	<code>result.peaks_by_property == {}</code>	Pass	0.001s
Empty property windts	<code>result.peaks_by_property == {}</code>	Pass	0.001s
Falls back to stem when event id missing	<code>result.event_id == "EVT-FALLBACK"</code>	Pass	0.001s
Falls back to stem when event id missing	<code>result.event_id == "EVT-FALLBACK"</code>	Pass	0.001s
Loads per property series when present	<code>result.series_by_property["PROP-a"] == ([0.0, 1.0, 2.0], ...; "PROP-b" not in result.series_by_property)</code>	Pass	0.001s
Loads per property series when present	<code>result.series_by_property["PROP-a"] == ([0.0, 1.0, 2.0], ...; "PROP-b" not in result.series_by_property)</code>	Pass	0.001s
Missing file raises	—	Pass	0.000s
Missing file raises	—	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Round trip from disk	<code>isinstance(result, EventWindts);</code> <code>result.event_id == "EVT-0001"</code> (+4 more)	Pass	0.001s
Round trip from disk	<code>isinstance(result, EventWindts);</code> <code>result.event_id == "EVT-0001"</code> (+4 more)	Pass	0.001s
Series defaults empty for legacy	<code>result.series_by_property == {}</code>	Pass	0.001s
Series defaults empty for legacy	<code>result.series_by_property == {}</code>	Pass	0.001s
Series skipped when lengths mismatch	<code>result.peaks_by_property</code> <code>== {"PROP-a": 40.0};</code> <code>result.series_by_property == {}</code>	Pass	0.001s
Series skipped when lengths mismatch	<code>result.peaks_by_property</code> <code>== {"PROP-a": 40.0};</code> <code>result.series_by_property == {}</code>	Pass	0.001s
Skips entries with missing id	<code>result.peaks_by_property ==</code> <code>{"PROP-a": 32.4}</code>	Pass	0.001s
Skips entries with missing id	<code>result.peaks_by_property ==</code> <code>{"PROP-a": 32.4}</code>	Pass	0.001s
Skips entries with missing peak	<code>result.peaks_by_property ==</code> <code>{"PROP-a": 32.4}</code>	Pass	0.001s
Skips entries with missing peak	<code>result.peaks_by_property ==</code> <code>{"PROP-a": 32.4}</code>	Pass	0.001s
Bounded between floor and one	<code>WIND_PERSISTENCE_GUST_FLOOR <= phi <=</code> <code>1.0</code>	Pass	0.000s
Bounded between floor and one	<code>WIND_PERSISTENCE_GUST_FLOOR <= phi <=</code> <code>1.0</code>	Pass	0.000s
Monotonic increasing in duration	<code>b >= a</code>	Pass	0.000s
Monotonic increasing in duration	<code>b >= a</code>	Pass	0.000s
Negative duration is gust floor	<code>persistence_factor(-3.0) ==</code> <code>pytest.approx(WIND_PERSISTENC...</code>	Pass	0.000s
Negative duration is gust floor	<code>persistence_factor(-3.0) ==</code> <code>pytest.approx(WIND_PERSISTENC...</code>	Pass	0.000s
Saturates to one for long duration	<code>persistence_factor(1000.0) ==</code> <code>pytest.approx(1.0, abs=1e-6)</code>	Pass	0.000s
Saturates to one for long duration	<code>persistence_factor(1000.0) ==</code> <code>pytest.approx(1.0, abs=1e-6)</code>	Pass	0.000s
Value at tau matches formula	<code>phi == pytest.approx(expected,</code> <code>abs=1e-12)</code>	Pass	0.000s
Value at tau matches formula	<code>phi == pytest.approx(expected,</code> <code>abs=1e-12)</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Zero duration is gust floor	<code>persistence_factor(0.0) ==</code> <code>pytest.approx(WIND_PERSISTENCE...</code>	Pass	0.000s
Zero duration is gust floor	<code>persistence_factor(0.0) ==</code> <code>pytest.approx(WIND_PERSISTENCE...</code>	Pass	0.000s
Brief gust less than sustained	<code>d.brief < d.long</code>	Pass	0.000s
Brief gust less than sustained	<code>d.brief < d.long</code>	Pass	0.000s
Duration measured above v50 eff	<code>d.hard["duration_above_hours"]</code> <code>< d.soft["duration_above_h...;</code> <code>d.hard["damage_ratio"] <</code> <code>d.soft["damage_ratio"]</code>	Pass	0.000s
Duration measured above v50 eff	<code>d.hard["duration_above_hours"]</code> <code>< d.soft["duration_above_h...;</code> <code>d.hard["damage_ratio"] <</code> <code>d.soft["damage_ratio"]</code>	Pass	0.000s
No series falls back to peak only	<code>d["duration_above_hours"] is None;</code> <code>d["persistence_factor"] == 1.0</code> (+1 more)	Pass	0.000s
No series falls back to peak only	<code>d["duration_above_hours"] is None;</code> <code>d["persistence_factor"] == 1.0</code> (+1 more)	Pass	0.000s
Run event round trip includes persistence	<code>d["duration_above_hours"] ==</code> <code>pytest.approx(5.0, abs=1e-9);</code> 0.45 <code><= d["persistence_factor"] < 1.0</code> (+1 more)	Pass	0.001s
Run event round trip includes persistence	<code>d["duration_above_hours"] ==</code> <code>pytest.approx(5.0, abs=1e-9);</code> 0.45 <code><= d["persistence_factor"] < 1.0</code> (+1 more)	Pass	0.002s
Series adds duration and persistence fields	<code>d["duration_above_hours"] ==</code> <code>pytest.approx(6.0, abs=1e-9);</code> 0.45 <code><= d["persistence_factor"] < 1.0</code>	Pass	0.000s
Series adds duration and persistence fields	<code>d["duration_above_hours"] ==</code> <code>pytest.approx(6.0, abs=1e-9);</code> 0.45 <code><= d["persistence_factor"] < 1.0</code>	Pass	0.000s
Empty record uses default	<code>v_50 ==</code> <code>pytest.approx(DEFAULT_WIND_THRESHOLD_KPH</code> <code>/ 3.6, a...</code>	Pass	0.000s
Empty record uses default	<code>v_50 ==</code> <code>pytest.approx(DEFAULT_WIND_THRESHOLD_KPH</code> <code>/ 3.6, a...</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Falls back to default when missing	<code>v_50 ==</code> <code>pytest.approx(DEFAULT.WIND.THRESHOLD_KPH</code> <code>/ 3.6, a...</code>	Pass	0.000s
Falls back to default when missing	<code>v_50 ==</code> <code>pytest.approx(DEFAULT.WIND.THRESHOLD_KPH</code> <code>/ 3.6, a...</code>	Pass	0.000s
Falls back when field value invalid	<code>v_50 ==</code> <code>pytest.approx(DEFAULT.WIND.THRESHOLD_KPH</code> <code>/ 3.6, a...</code>	Pass	0.000s
Falls back when field value invalid	<code>v_50 ==</code> <code>pytest.approx(DEFAULT.WIND.THRESHOLD_KPH</code> <code>/ 3.6, a...</code>	Pass	0.000s
Returns float in meters per second	<code>isinstance(v_50, float); 25.0 < v_50 <</code> <code>45.0</code>	Pass	0.000s
Returns float in meters per second	<code>isinstance(v_50, float); 25.0 < v_50 <</code> <code>45.0</code>	Pass	0.000s
Uses cdm field when present	<code>v_50 == pytest.approx(120.0 / 3.6,</code> <code>abs=1e-9)</code>	Pass	0.000s
Uses cdm field when present	<code>v_50 == pytest.approx(120.0 / 3.6,</code> <code>abs=1e-9)</code>	Pass	0.000s
Run event directory writes one file per input	<code>[p.name for p in written] ==</code> <code>["EVT-0000.json", "EVT-0001....;</code> <code>p.exists()</code>	Pass	0.003s
Run event directory writes one file per input	<code>[p.name for p in written] ==</code> <code>["EVT-0000.json", "EVT-0001....;</code> <code>p.exists()</code>	Pass	0.003s
Writes json to output path	<code>out_path.exists(); on_disk == payload</code> <code>(+2 more)</code>	Pass	0.003s
Writes json to output path	<code>out_path.exists(); on_disk == payload</code> <code>(+2 more)</code>	Pass	0.002s
At base v50 yields half	<code>v == pytest.approx(0.5, abs=1e-9)</code>	Pass	0.000s
At base v50 yields half	<code>v == pytest.approx(0.5, abs=1e-9)</code>	Pass	0.000s
Far above threshold saturates at one	<code>v == pytest.approx(1.0, abs=1e-6); v</code> <code><= 1.0</code>	Pass	0.000s
Far above threshold saturates at one	<code>v == pytest.approx(1.0, abs=1e-6); v</code> <code><= 1.0</code>	Pass	0.000s
Far below threshold approaches zero	<code>v == pytest.approx(0.0, abs=1e-6)</code>	Pass	0.000s
Far below threshold approaches zero	<code>v == pytest.approx(0.0, abs=1e-6)</code>	Pass	0.000s
Monotonic increasing	<code>b >= a</code>	Pass	0.000s
Monotonic increasing	<code>b >= a</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Negative wind yields zero	<code>scalar_peak_damage(-5.0) == 0.0;</code> <code>bri_peak_damage(-5.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Negative wind yields zero	<code>scalar_peak_damage(-5.0) == 0.0;</code> <code>bri_peak_damage(-5.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Slope at midpoint matches config	<code>slope == pytest.approx(expected, rel=1e-3)</code>	Pass	0.000s
Slope at midpoint matches config	<code>slope == pytest.approx(expected, rel=1e-3)</code>	Pass	0.000s
Symmetric around threshold	<code>(a + b) == pytest.approx(1.0, abs=1e-9)</code>	Pass	0.000s
Symmetric around threshold	<code>(a + b) == pytest.approx(1.0, abs=1e-9)</code>	Pass	0.000s
Zero wind yields zero damage	<code>scalar_peak_damage(0.0) == 0.0;</code> <code>bri_peak_damage(0.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Zero wind yields zero damage	<code>scalar_peak_damage(0.0) == 0.0;</code> <code>bri_peak_damage(0.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Equals peak curve times persistence	<code>scalar_wind_damage(v, t) == pytest.approx(sc...</code>	Pass	0.000s
Equals peak curve times persistence	<code>scalar_wind_damage(v, t) == pytest.approx(sc...</code>	Pass	0.000s
Never exceeds one	<code>scalar_wind_damage(200.0, 10_000.0) <= 1.0</code>	Pass	0.000s
Never exceeds one	<code>scalar_wind_damage(200.0, 10_000.0) <= 1.0</code>	Pass	0.000s
Saturates to peak curve	<code>scalar_wind_damage(v, 10_000.0) == pytest.approx(...</code>	Pass	0.000s
Saturates to peak curve	<code>scalar_wind_damage(v, 10_000.0) == pytest.approx(...</code>	Pass	0.000s
Single gust is less than sustained	<code>gust < sustained</code>	Pass	0.000s
Single gust is less than sustained	<code>gust < sustained</code>	Pass	0.000s
Zero wind yields zero	<code>scalar_wind_damage(0.0, 5.0) == 0.0;</code> <code>bri_wind_damage(0.0, 5.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s
Zero wind yields zero	<code>scalar_wind_damage(0.0, 5.0) == 0.0;</code> <code>bri_wind_damage(0.0, 5.0, threshold_ms=30.0) == 0.0</code>	Pass	0.000s

7.4 Test Document History

Date	Version	Description	Author
28-May-2026	1.0	Initial test suite (132 tests)	D. Kelly
03-Jun-2026	1.1	12 test(s) added (total: 78)	D. Kelly
16-Jun-2026	1.2	19 test(s) added (total: 97)	D. Kelly
21-Jul-2026	1.3	27 test(s) added (total: 124)	D. Kelly

8 Model Limitations and Known Weaknesses

WD-L1 — Uncalibrated resilience cap. $\delta_{\max} = 5$ m/s is large relative to the logistic’s transition width ($1/a = 5$ m/s). A capped shift therefore moves the curve by a full transition width, changing mid-range damage by a factor of several. The credit the model awards is thus highly sensitive to a parameter with no empirical basis.

WD-L2 — Sustained wind, no gust conversion. The curve is defined on sustained wind but real damage is driven substantially by gusts. The implied gust factor is folded into the seed calibration of v_{50} rather than modelled, so any change to the wind-field model’s averaging convention would silently invalidate the curve.

WD-L3 — Isotropic vulnerability. Direction of attack, building orientation, and shielding by neighbouring structures are not represented.

WD-L4 — Duration measured against a moving threshold. The persistence term is defined as hours above the damage-onset threshold, which is itself BRI-adjusted. A resilient building therefore accrues fewer hours as well as a lower severity, so the resilience credit compounds across the two terms. This is intentional but means the two effects cannot be attributed separately from the output alone.

WD-L5 — Independence from flood damage. Wind and flood damage to the same property in the same event are computed independently and, where combined downstream, may double-count damage to shared building elements.

WD-L6 — Binary trigger discards near-misses. A property whose peak wind falls one m/s short of its effective threshold contributes nothing to the PRS count, while one m/s above contributes fully. The spread is therefore discontinuous in wind at the threshold, by design, but this makes portfolio PRS sensitive to small shifts in the wind field near the threshold band.

WD-L7 — Grade mid-points for commercial assets. Commercial assets carry letter-grade envelopes rather than numeric sub-scores; using band mid-points understates within-grade dispersion.

8.1 Remediation

WD-R1 Calibrate a , v_{50} and the persistence pair against observed wind loss; precondition of sign-off.

WD-R2 Calibrate or justify $\delta_{\max}$, α_w and β_c against resilience-stratified loss experience (addresses WD-L1).

WD-R3 Make the averaging convention explicit in the interface, so a change upstream cannot silently invalidate the curve (WD-L2).

WD-R4 Define and document the treatment of combined wind-and-flood damage to a single property (WD-L5).

9 Governance Validation Questions

The following nine questions are mandated by Chapter 5 of the *Model Risk Governance for Vendors* handbook and must be explicitly addressed for every model in the inventory. Response status is tracked in the Model Governance Dashboard (MRC portal).

9.1 Q1 — Model purpose and use

“Converts per-property peak sustained wind into a damage ratio for loss reporting, and into a binary damage-onset trigger for PRS event counting. Used by the wind-impact route and by the PRS pricer’s wind leg.”

9.2 Q2 — Conceptual soundness

“A logistic vulnerability curve is standard practice for wind. The log-differential resilience shift mirrors the already-reviewed flood stilt, and the saturating persistence factor is a conventional duration-of-load treatment. The peak-only curves being the exact $t \rightarrow \infty$ limit of the persistence-aware curves is a structural consistency guarantee.”

9.3 Q3 — Data quality

“Peak wind and duration come from the wind time-series stage; thresholds and BRI scores from the CDM. Missing BRI scores default to the neutral reference, and a missing threshold to a configured default — both are documented no-information defaults rather than failures, which means data gaps degrade quietly. Coverage of the CDM wind-threshold field should be monitored.”

9.4 Q4 — Assumptions

“Sustained wind is the correct severity measure (WD-L2); vulnerability is isotropic (WD-L3); damage accumulates with exposure time on a 4-hour scale; wind and flood damage are independent (WD-L5).”

9.5 Q5 — Implementation verification

“Unit tests cover all four curves, the shift including its cap and log floor, the persistence limits, all four threshold-resolution branches, the CDM navigation helpers, the extraction layer and the event composer.”

9.6 Q6 — Sensitivity and stability

“See Section 6. The model is monotone and materially sensitive along all three axes, with the resilience credit concentrated in the 20–40 m/s band by design. Stability is not a concern — the model is a deterministic closed-form evaluation with no sampling.”

9.7 Q7 — Limitations and weaknesses

“Documented in Section 8 (WD-L1–WD-L7). WD-L1 is the most material: the resilience credit rests on an uncalibrated cap comparable to the curve’s own transition width.”

9.8 Q8 — Ongoing monitoring

“Not yet in production. The plan under WD-R1 covers CDM threshold-field coverage, the distribution of realised triggers per event, and a backtest of predicted against realised wind loss once data exists.”

9.9 Q9 — Governance and approval

“Owner: David K Kelly. Status: Development, Tier 2, RAG Amber. Not approved for production; sign-off is contingent on closing WD-R1–WD-R4.”

10 Change History

Version	Date	Description
1.0	21-July-2026	Initial model documentation: severity, resilience and persistence framework, seed para